

MVP 2 — Produção de Óleo e Gás a partir de dados da ANP ao longo do tempo

Autor: Felipe Barreto R. S. Macedo **Data:** 26/09/2026 **Matrícula** 4052026000629

Sprint 2 - Engenharia de Dados

Link GitHub: https://github.com/felipebrsm/PUC_Sprint_2_Databricks_felipebrsm

1. Motivação e objetivo

Esse trabalho é o MVP da pós-graduação em Ciência de Dados, Sprint 2, Engenharia de Dados.

Como funcionário da Petrobras, escolhi um tema que abordasse dados relativos à minha área de interesse. Para isso, propus uma análise de produção de petróleo e gás natural a partir de dados públicos da Agência Nacional do Petróleo (ANP). Foi utilizado o modelo medallion, com a criação de camadas Bronze, Silver e Gold, por meio do Databricks, e usar essa estrutura para responder perguntas que eu mesmo formulei. A análise foi focada em duas frentes: técnica (quanto se produz, onde, como evolui) e econômica (como isso se relaciona com preço do petróleo e câmbio).

2. Fontes de dados

Combinei cinco fontes diferentes, três da própria ANP e duas de indicadores econômicos:

Fonte	Informação
BAR — Boletim Anual de Recursos e Reservas	Um arquivo por ano, com o volume de óleo e gás estimado em cada campo, o quanto já foi produzido de forma acumulada, a fração já recuperada da reserva e se o campo está classificado como maduro ou não. Apresenta o total ainda em campo.
BMP — Boletim Mensal de Produção	A fonte com mais linhas do projeto: um registro por poço, por mês, com produção de óleo, gás, água e volumes injetados (água, gás, CO ₂ etc.). Apresenta o total que saiu do campo.
BDEP — Poços Perfurados Públicos	Lista de poços cujos dados técnicos já foram tornados públicos pela ANP: localização (latitude/longitude), operador, profundidade, situação, se é terra ou mar. Apresenta as características técnicas e espaciais do poço.
Câmbio USD/BRL	Cotação diária do dólar frente ao real, baixada da API do Banco Central (SGS). Agregada por mês na camada Gold.
Brent	Preço diário do petróleo tipo Brent (referência internacional), baixado da API do EIA (agência de energia dos EUA). Também agregado por mês.

Optei por um esquema em estrela (star schema) na camada Gold, com três dimensões e três tabelas de fato compartilhando essas dimensões. A tabela abaixo resume o resultado final:

Tabela	Comentário
dim_campo	Chave: nome do campo + bacia (para evitar campos homônimos em bacias diferentes) Quantidade de entradas: 590 combinações.
dim_poco	Chave: poço Quantidade de entradas: 31.304)

dim_tempo	Um mês do calendário (ano, mês, trimestre).
fact_producao_mensal	Produção de cada poço, em cada mês (aprox. 5,1 milhões de linhas)
fact_reservas_anual	Reserva estimada e fração recuperada de cada campo, por ano (e por situação: maduro ou não).
fact_economico	Câmbio e Brent médios de cada mês.

3. Perguntas Propostas

- A) Qual foi a produção total de óleo e gás natural por ano?
- B) Quais os campos com maior produção acumulada?
- C) Qual a distribuição de poços por bacia?
- D) Há correlação entre aumento do Brent e aumento da produção? Essa correlação se mantém para alguns meses depois?
- E) Qual seria a receita bruta estimada (produção × Brent × câmbio) por campo e por bacia?

4. Estrutura dos notebooks e problemas encontrados

O projeto ficou dividido em quatro notebooks no Databricks: três seguindo o padrão medallion (Bronze, Silver, Gold) e um quarto (Análise) só para responder as perguntas em cima da Gold já pronta.

A maior dificuldade do projeto foi encontrar e corrigir problemas de qualidade nos dados brutos da ANP, que não apareciam de forma óbvia. Muitas vezes a correção só foi feita depois de eu confirmar o problema com uma consulta de diagnóstico antes de mexer no código.

Os notebooks apresentados como entrega são as versões consolidadas, após as diversas rodadas de tentativa e erro.

1) Preparação do Databricks

Criei um novo catálogo chamado PUC_Sprint_2, com um único schema dentro dele "anp". Foram criados três Volumes nesse schema para subir arquivo bruto direto pela interface.

O raw guarda os arquivos originais baixados da ANP, do BCB e do EIA, sem nenhuma transformação, organizados em subpastas por fonte (bmp, bar, bdep, economico, e depois bmp-2024 pros arquivos que faltaram no scrape original). O checkpoints ficou reservado para metadado de processamento incremental, e o exports para eventuais saídas do projeto, que acabaram não sendo utilizados.

Para persistência das tabelas usei Delta Tables, também dentro do schema anp. Cada notebook grava seu próprio conjunto: a Bronze grava bronze_bmp, bronze_bar, bronze_bdep, bronze_cambio e bronze_brent lendo do Volume raw; a Silver grava silver_bmp, silver_bar, silver_bdep, silver_cambio e silver_brent lendo as tabelas Bronze; e a Gold grava o modelo final (dim_campo, dim_poco, dim_tempo, fact_producao_mensal, fact_reservas_anual, fact_economico) lendo das tabelas Silver. Em todos os casos uso DROP TABLE IF EXISTS antes de gravar de novo, então rodar o pipeline do zero sempre deixa as tabelas num estado limpo.

Catalog	Name	Owner	Created at	Popularity
Serverless Starter Wareho... Serverless 2XS	bronze_bar	felipebrsm@gmail.com	Sep 24, 2026, 06:13 ...	...
Catalog	bronze_bdep	felipebrsm@gmail.com	Sep 24, 2026, 06:13 ...	...
Type to search	bronze_bmp	felipebrsm@gmail.com	Sep 24, 2026, 06:13 ...	...
My organization	bronze_brent	felipebrsm@gmail.com	Sep 24, 2026, 06:13 ...	...
workspace	bronze_cambio	felipebrsm@gmail.com	Sep 24, 2026, 06:13 ...	...
system	dim_campo	felipebrsm@gmail.com	Sep 24, 2026, 06:24 ...	...
puc_sprint_2	dim_poco	felipebrsm@gmail.com	Sep 24, 2026, 06:24 ...	...
anp	dim_tempo	felipebrsm@gmail.com	Sep 24, 2026, 06:24 ...	...
Tables (16)	fact_economico	felipebrsm@gmail.com	Sep 24, 2026, 06:24 ...	...
Volumes (3)	fact_producao_mensal	felipebrsm@gmail.com	Sep 24, 2026, 06:24 ...	...
checkpoints	fact_reservas_anual	felipebrsm@gmail.com	Sep 24, 2026, 06:24 ...	...
exports	silver_bar	felipebrsm@gmail.com	Sep 24, 2026, 06:14 ...	...
raw	silver_bdep	felipebrsm@gmail.com	Sep 24, 2026, 06:15 ...	...
default	silver_bmp	felipebrsm@gmail.com	Sep 24, 2026, 06:14 ...	...
information_schema	silver_brent	felipebrsm@gmail.com	Sep 24, 2026, 06:15 ...	...
Shares received	silver_cambio	felipebrsm@gmail.com	Sep 24, 2026, 06:15 ...	...
samples				

2) Raw

A primeira etapa foi a mineração dos dados do site da ANP.

- BMP e BAR: escrevi um script em Python que lê a página de "dados abertos" da ANP e usa BeautifulSoup para achar os links de arquivo que batem com um padrão, em vez de montar a URL de cada arquivo "na mão". O script também segue a paginação da página para não perder arquivos que estão em páginas seguintes da listagem.

```
import os
import re
from urllib.parse import urljoin

import requests
from bs4 import BeautifulSoup

BMP_PAGE = "https://www.gov.br/anp/pt-br/centrais-de-conteudo/dados-abertos/fase-de-desenvolvimento-e-producao"
# Página de listagem completa (com paginação) - traz mais anos que a página
# "landing" de reservas-nacionais-de-petroleo-e-gas-natural.
BAR_PAGE = "https://www.gov.br/anp/pt-br/centrais-de-conteudo/dados-estatisticos/arquivos-reservas-nacionais-de-petroleo-e-gas-natural"

HEADERS = {"User-Agent": "Mozilla/5.0 (compatible; MVP-datapipeline/1.0)"}
```

```
def scrape_links(page_url: str, pattern: str, max_pages: int = 10) -> list[str]:
    links: set[str] = set()
    url = page_url
    for _ in range(max_pages):
        resp = requests.get(url, headers=HEADERS, timeout=30)
        resp.raise_for_status()
        soup = BeautifulSoup(resp.text, "html.parser")
        for a in soup.find_all("a", href=True):
            if re.search(pattern, a["href"], re.IGNORECASE):
                links.add(urljoin(url, a["href"]))
        next_link = soup.find("a", string=re.compile(r"Próximo"))
        if not next_link:
            break
        url = urljoin(url, next_link["href"])
    return sorted(links)

def download_new(urls: list[str], dest_dir: str) -> None:
    os.makedirs(dest_dir, exist_ok=True)
    for url in urls:
        # Links de listagem (Plone) terminam em "/view" em vez do nome do
        # arquivo com extensão - remove esse sufixo antes de extrair o nome.
        clean_url = re.sub(r"/view/?$", "", url)
        filename = clean_url.split("/")[-1].split("?")[0]
        dest_path = os.path.join(dest_dir, filename)
        if os.path.exists(dest_path):
            print(f"já existe {filename}")
            continue
        print(f"baixando {filename}")
        r = requests.get(clean_url, headers=HEADERS, timeout=60)
        r.raise_for_status()
```

```

if __name__ == "__main__":
    # BMP: pega todos os csv de produção (mar e terra, todos os anos listados)
    bmp_links = scrape_links(BMP_PAGE, r"producao.*\.csv$")
    print(f"{len(bmp_links)} arquivos de BMP encontrados na página.")
    download_new(bmp_links, "downloads/bmp")

    # BAR: pega os xlsx "tabela-dados-bar-(ano).xlsx" (o link na listagem
    # termina em "/view", por isso sem âncora $ no regex)
    bar_links = scrape_links(BAR_PAGE, r"tabela-dados-bar.*\.xlsx")
    print(f"{len(bar_links)} arquivos de BAR encontrados na página.")
    download_new(bar_links, "downloads/bar")

```

- BDEP: a tabela completa de poços só existe atrás de um aplicativo interativo da ANP, sem CSV direto — por isso usei último dataset disponível: "Poços Perfurados Públicos", acumulando dados de aproximadamente 31.000 poços.
- Câmbio USD/BRL: baixado via API SGS do Banco Central. Essa API só aceita períodos de até 10 anos por consulta, então o script pede os dados em janelas de 10 em 10 anos e depois junta tudo.

```

import requests, pandas as pd

def baixar_cambio(data_inicio: str, data_fim: str, serie: int = 1) -> pd.DataFrame:
    """Baixa uma janela (máx. 10 anos) da série do BCB SGS."""
    url = f"https://api.bcb.gov.br/dados/serie/bcdata.sgs.{serie}/dados?formato=json&dataInicial={data_inicio}"
    resp = requests.get(url, timeout=30)
    dados = resp.json()
    if isinstance(dados, dict): # API retornou erro em vez de lista
        raise RuntimeError(f"Erro da API BCB: {dados.get('error', dados)}")
    return pd.DataFrame(dados)

# Janelas de até 10 anos cada, cobrindo 2010–2025
janelas = [("01/01/2010", "31/12/2019"), ("01/01/2020", "31/12/2025")]
partes = [baixar_cambio(ini, fim) for ini, fim in janelas]
cambio = pd.concat(partes, ignore_index=True)
import os

os.makedirs("downloads/economico", exist_ok=True)
cambio.to_csv("downloads/economico/cambio_usd_brl.csv", index=False)
cambio.to_csv("downloads/economico/cambio_usd_brl.csv", index=False)
print(f"{len(cambio)} registros de câmbio baixados.")

```

- Brent: baixado via API v2 do EIA (agência de energia dos EUA), que exige uma chave de acesso gratuita

```

import pandas as pd
import requests

API_KEY = "XXXXXX"
BASE_URL = "https://api.eia.gov/v2/petroleum/pri/spt/data/"
PAGE_SIZE = 5000 # máximo permitido por página na API v2 do EIA

def baixar_brent(api_key: str) -> pd.DataFrame:
    if not api_key:
        raise RuntimeError("API key não fornecida")

    registros = []
    offset = 0
    while True:
        params = {
            "api_key": api_key,
            "frequency": "daily",
            "data[0]": "value",
            "facets[series][0]": "RB RTE",
            "sort[0][column]": "period",
            "sort[0][direction]": "asc",
            "offset": offset,
            "length": PAGE_SIZE,
        }
        resp = requests.get(BASE_URL, params=params, timeout=60)
        resp.raise_for_status()
        payload = resp.json()

        if "response" not in payload or "data" not in payload["response"]:
            raise RuntimeError(f"Resposta inesperada da API do EIA: {payload}")

        pagina = payload["response"]["data"]

```

```

registros.extend(pagina)
print(f"[baixado] offset={offset} -> {len(pagina)} registros")

if len(pagina) < PAGE_SIZE:
    break # última página
offset += PAGE_SIZE

return pd.DataFrame(registros)

if __name__ == "__main__":
    os.makedirs("downloads/economico", exist_ok=True)

    brent = baixar_brent(API_KEY)
    destino = "downloads/economico/brent_spot.csv"
    brent.to_csv(destino, index=False)
    print(f"Concluído: {len(brent)} registros salvos em {destino}")

```

Um problema que só apareceu bem mais tarde, já trabalhando na Bronze: alguns arquivos recentes (2024 e parte de 2025) foram publicados pela ANP com um nome de arquivo diferente do padrão que o script original reconhecia, então ficaram de fora do primeiro download. Precisei localizar esses arquivos manualmente na página da ANP e adicionar em uma pasta separada (raw/bmp-2024) para a Bronze incorporar.

Os arquivos foram importados nas respectivas pastas RAW, criadas nos volumes do Databricks.

The screenshot shows the Databricks file browser interface for a volume named 'raw'. The top navigation bar includes a folder icon, the name 'raw', and icons for share and star. A 'Share' dropdown and an 'Upload to this volume' button are also present. Below the navigation bar, there are tabs for 'Overview', 'Files', 'Details', 'Permissions', and 'Policies'. The 'Overview' tab is active, showing a 'Description' section with 'AI generate' and 'Add' buttons. The main content area displays the path '/Volumes/puc_sprint_2/anp/raw' with 'Refresh' and 'Create directory' buttons. A search bar is located below the path. A table lists the contents of the directory:

Name	Size	Last modified
bar		
bdep		
bmp-2024		
bmp		
bmp_corrigido		
economico		

3) Bronze

Para a camada Bronze, busquei consumir dos volumes RAW, implementando ajustes necessários apenas para garantir a leitura correta da ordem das tabelas nos diferentes arquivos de cada fonte e mexer o mínimo possível no dado, só o suficiente para o Delta aceitar a tabela (nome de coluna sem caracteres proibido). Mesmo assim, apareceram problemas nos arquivos da ANP.

- Separador e encoding diferentes entre fontes (BDEP usa ";" e Latin-1; BAR usa "," e UTF-8).

```
def sanitizar_para_delta(df):
    novos_nomes = [re.sub(r"[ ,;{}()\[\]\n\t=]+", "_", c).strip("_") for c in df.columns]
    return df.toDF(*novos_nomes)
```

- ~345 mil linhas do BMP (7% do total, concentradas em 15 arquivos) vinham com a linha inteira encapsulada entre aspas duplicadas.

```
def desfazer_encapsulamento_duplo(linha: str) -> str:
    linha = linha.rstrip("\r\n")
    if linha.startswith('"') and linha.endswith('"'):
        linha = linha[1:-1].replace('""', '"')
    return linha + "\n"
```

- Precisei detectar e desfazer isso arquivo a arquivo, porque cada um tinha um encoding diferente (alguns Latin-1, outros UTF-8 com marca de BOM).

```
def ler_texto_com_encoding_automatico(caminho: str) -> str:
    with open(caminho, "rb") as f:
        dados_brutos = f.read()
    try:
        return dados_brutos.decode("utf-8-sig")
    except UnicodeDecodeError:
        return dados_brutos.decode("ISO-8859-1")
```

- O ano de 2024 não tinha sido baixado — a ANP publicou esse ano com um nome de arquivo diferente do padrão, então o script original de download não achou. Precisei localizar manualmente e incluir.
- um arquivo de 2025 tinha a ORDEM das colunas trocada em relação a todos os outros. Como eu lia vários arquivos de uma vez só, o Spark alinhava por posição, não por nome, isso fez valores de campo, estado e instalação ficarem embaralhados, sem erro nenhum aparecer. Corrigi reescrevendo a ingestão do BMP para ler cada arquivo individualmente e renomear as colunas pelo nome real (não pela posição), o que também deixa o pipeline protegido contra esse tipo de bug no futuro.

```
def padronizar_colunas_bmp(df, identificador_arquivo=""):
    mapa_rename = {}
    for coluna_original in df.columns:
        chave = chave_normalizada(coluna_original)
        if chave not in MAPA_COLUNAS_BMP:
            raise ValueError(
                f"Coluna do BMP não reconhecida em {identificador_arquivo!r}: "
                f"{coluna_original!r} (chave normalizada: {chave!r}). "
                f"Adicione essa variação ao MAPA_COLUNAS_BMP antes de prosseguir."
            )
        mapa_rename[coluna_original] = MAPA_COLUNAS_BMP[chave]

    for original, novo in mapa_rename.items():
        df = df.withColumnRenamed(original, novo)
    return df.select(*ORDEM_CANONICA_BMP)
```

- Para todas as tabelas, foram utilizadas as funções de ajuste, correção e saneamento necessárias:

```
RAW_BAR = f"/Volumes/{CATALOGO}/{SCHEMA}/raw/bar"

df_bar = (spark.read
    .option("header", True)
    .option("sep", ",")
    .option("encoding", "UTF-8")
    .csv(f"{RAW_BAR}/*.csv"))

df_bar_bronze = sanitizar_para_delta(df_bar)
df_bar_bronze.printSchema()
```

- Ao final, as tabelas Bronze foram salvas, sobrescrevendo os schemas. Um último teste foi criada para verificar a existência.

- Texto com acentuação corrompida (mojibake) no BAR, por um problema de encoding na origem. Houve problemas em particular com a letra "Ã", que surgiu às vezes como mojibake, às vezes como letra real, no campo de "GAVIÃO", o que impactou no notebook de análise.

```
def corrigir_mojibake_colunas(df, colunas):
    for c in colunas:
        candidato = F.decode(F.encode(F.col(c), "ISO-8859-1"), "UTF-8")
        usar_candidato = F.col(c).rlike("[ÃÀ]") & ~candidato.contains("\uFFFF")
        df = df.withColumn(c, F.when(usar_candidato, candidato).otherwise(F.col(c)))
    return df
```

- Percebi, mais tarde, que a Silver implementou por conta própria uma limpeza que já tinha sido feita na Bronze — duas cópias da mesma lógica em lugares diferentes. Isso é arriscado (uma correção feita só num lugar não se propaga), então simplifiquei para a Silver ler direto da tabela Bronze já corrigida.
- Após a verificação e ajustes nas planilhas BAR, BDEP, BMP e ECO, salvei as tabelas:

```
for tabela in ["silver_bmp", "silver_bar", "silver_bdep", "silver_cambio", "silver_brent"]:
    spark.sql(f"DROP TABLE IF EXISTS PUC_Sprint_2.anp.{tabela}")

df_bmp_silver.write.mode("overwrite").saveAsTable("PUC_Sprint_2.anp.silver_bmp")
df_bar_silver.write.mode("overwrite").saveAsTable("PUC_Sprint_2.anp.silver_bar")
df_bdep_silver.write.mode("overwrite").saveAsTable("PUC_Sprint_2.anp.silver_bdep")
df_cambio_silver.write.mode("overwrite").saveAsTable("PUC_Sprint_2.anp.silver_cambio")
df_brent_silver.write.mode("overwrite").saveAsTable("PUC_Sprint_2.anp.silver_brent")

for tabela in ["silver_bmp", "silver_bar", "silver_bdep", "silver_cambio", "silver_brent"]:
    existe = spark.catalog.tableExists(f"PUC_Sprint_2.anp.{tabela}")
    print(f"{tabela}: {'existe' if existe else 'AINDA NÃO EXISTE'}")
```

5) Gold

Com os dados limpos e salvos em suas respectivas tabelas Silver, foi criada a camada Gold. Escolhi o modelo Estrela, na forma de uma constelação de fatos, porque o projeto tem três processos de negócio distintos: produção por poço, reservas por campo e indicadores econômicos (câmbio e Brent). Cada um tem granularidade própria, então criei uma tabela fato para cada um, todas compartilhando as dimensões campo, poço e tempo. Esse compartilhamento evita duplicar informações descritivas entre os fatos e permite cruzar dados de fatos diferentes através da dimensão tempo. Um esquema mais simples deixa as consultas analíticas mais rápidas e diretas, com as métricas concentradas nos fatos e as descrições nas dimensões.

Cada tabela de dimensão e de fato só foi criada depois de eu confirmar, com uma consulta, que a suposta "chave única" realmente era única.

- **dim_campo**: nome de campo repetido em bacias diferentes, resolvido usando campo+bacia como chave composta.

```
# 1a. Diagnóstico: quantos campos têm bacia/estado inconsistente
conflitos_campo = (df_bmp
    .groupBy("campo")
    .agg(F.countDistinct("bacia").alias("qtd_bacias"), F.countDistinct("estado").alias("qtd_estados"))
    .filter((F.col("qtd_bacias") > 1) | (F.col("qtd_estados") > 1)))

print(f"Campos com bacia/estado inconsistente: {conflitos_campo.count()}")

> See performance \(1\) ✓ Checked for improvement
> conflitos_campo: pyspark.sql.connect.dataframe.DataFrame = [campo: string, qtd_bacias: long ... 1 more field]
Campos com bacia/estado inconsistente: 8
```

```
▶ 2 days ago (2s) 5

# 1b. Resolve o estado "vencedor" por (campo, bacia) usando o critério de maior produção histórica
janela_campo_bacia = Window.partitionBy("campo", "bacia").orderBy(F.desc("producao_total"))

estado_dominante = (df_bmp
    .groupBy("campo", "bacia", "estado")
    .agg(F.sum("producao_oleo_m3").alias("producao_total"))
    .withColumn("rank", F.row_number().over(janela_campo_bacia))
    .filter(F.col("rank") == 1)
    .select("campo", "bacia", "estado"))

dim_campo = estado_dominante.withColumn("campo_id", F.monotonically_increasing_id())

# 1c. Validação: deve bater exatamente com o total de combinações (campo, bacia) distintas
print(f"dim_campo: {dim_campo.count()}")
print(f"combinações distintas (campo, bacia) no BMP: {df_bmp.select('campo', 'bacia').distinct().count()}")

> See performance \(2\) ✓ Checked for improvements

> 2 dataframes: dim_campo, estado_dominante
dim_campo: 590
combinações distintas (campo, bacia) no BMP: 590
```

- **dim_poço:** ao contrário do BAR, o BDEP não tem nenhuma duplicidade de poço (31.304 linhas = 31.304 poços distintos).
- **dim_tempo:** optou-se por usar o mês como chave, o que necessitou a agregação, ainda na camada Silver, da média de Brent e Câmbio. Outro problema:
 - a coluna de mês do BMP vem tanto em número ("12") quanto por extenso em português ("dez") para os mesmos meses recentes. A dimensão de tempo foi montada de forma errada no começo, juntando os valores distintos antes de calcular o mês, então "dez/2025" e "12/2025" viravam duas linhas diferentes representando o mesmo mês. Isso duplicava a produção de dezembro/2025 quando cruzada com a fato (cada linha de produção encontrava duas linhas de dimensão).

```
▶ 2 days ago (<1s) 10 Python

def extrair_mes_numerico(df, coluna_mes_ano="mes_ano"):
    mes_str = F.split(F.col(coluna_mes_ano), "/").getItem(0)
    mes_num = (F.when(mes_str == "out", F.lit(10))
               .when(mes_str == "nov", F.lit(11))
               .when(mes_str == "dez", F.lit(12))
               .otherwise(mes_str.try_cast("int")))
    return df.withColumn("mes", mes_num)
```

- **Fact_producao_mensal:** o join entre dim_campo e dim_poço indicou 987 linhas sem correspondência. Busquei avaliar a quantidade total produzida por essa seleção, que indicou apenas 7 poços específicos, ~4,1 milhões de m³ de um total de ~4,2 bilhões - 0,1% da produção. São poços recentes ainda não constantes no BDEP (que só lista poços já tornados públicos). Pela pouca proporção, aceitei como limitação.

```

fact_producao_mensal = (extrair_mes_numerico(df_bmp)
    .withColumn("tempo_id", (F.col("ano") * 100 + F.col("mes")).cast("int"))
    .join(dim_campo.select("campo", "bacia", "campo_id"), on=["campo", "bacia"], how="left")
    .join(dim_poco.select("poco_id"), df_bmp["poco"] == dim_poco["poco_id"], how="left")
    .select(
        "tempo_id", "campo_id", "poco_id",
        "producao_oleo_m3", "producao_condensado_m3", "producao_gas_associado_mm3",
        "producao_gas_nao_associado_mm3", "producao_agua_m3", "injecao_gas_mm3",
        "injecao_agua_recuperacao_secundaria_m3", "injecao_agua_descarte_m3",
        "injecao_gas_carbonico_mm3", "injecao_nitrogenio_mm3", "injecao_vapor_agua_t",
        "injecao_polimeros_m3", "injecao_outros_fluidos_m3",
    ))

# 4a. Validação: cobertura dos joins e ausência de tempo_id nulo
print(f"BMP original: {df_bmp.count()} | fact_producao_mensal: {fact_producao_mensal.count()}")
print(f"Sem campo_id (join campo/bacia falhou): {fact_producao_mensal.filter(F.col('campo_id').isNull()).count()}")
print(f"Sem poco_id (join poço falhou): {fact_producao_mensal.filter(F.col('poco_id').isNull()).count()}")
print(f"Sem tempo_id (mês não mapeado): {fact_producao_mensal.filter(F.col('tempo_id').isNull()).count()}")

> See performance \(5\) ✓ Checked for improvements

> fact_producao_mensal: pyspark.sql.connect.dataframe.DataFrame = [tempo_id: integer, campo_id: long ... 14 more fields]
BMP original: 5113367 | fact_producao_mensal: 5113367
Sem campo_id (join campo/bacia falhou): 0
Sem poco_id (join poço falhou): 987
Sem tempo_id (mês não mapeado): 0

```

- **fact_reservas_anual:** um mesmo campo podia ter duas linhas no mesmo ano (uma declaração de óleo maduro, outra de gás não maduro), resolvido incluindo a situação na chave. Alguns erros surgiram nessa etapa, com a falha em identificar campos como BÚZIOS e GAVIÃO por causa dos acentos. Esses erros foram corrigidos revisando o código das etapas Bronze e Silver.

```

fact_reservas_anual = (df_bar
    .join(dim_campo.select("campo", "bacia", "campo_id"), on=["campo", "bacia"], how="left")
    .select(
        "ano", "campo_id", "situacao",
        "voip_bbl", "vgip_m3", "petroleo_acumulado_bbl",
        "gas_natural_acumulado_m3", "fracao_recuperada_petroleo",
    ))

# 5c. Validação: contagem preservada (não deve colapsar nenhuma linha) e cobertura do join por VOIP
print(f"BAR original: {df_bar.count()} | fact_reservas_anual: {fact_reservas_anual.count()}")

campos_bar_sem_match = (df_bar.select("campo", "bacia").distinct()
    .join(dim_campo.select("campo", "bacia"), on=["campo", "bacia"], how="left_anti"))
voip_sem_match = (df_bar.join(campos_bar_sem_match, on=["campo", "bacia"], how="inner")
    .agg(F.sum("voip_bbl").collect()[0][0]))
voip_total = df_bar.agg(F.sum("voip_bbl").collect()[0][0])
print(f"Combinações campo+bacia sem match: {campos_bar_sem_match.count()}")
print(f"VOIP sem match: {voip_sem_match:.0f} bbl ({voip_sem_match/voip_total:.1%} do total)")

> See performance \(5\) ✓ Checked for improvements

> 2 dataframes: campos_bar_sem_match, fact_reservas_anual
BAR original: 2130 | fact_reservas_anual: 2130
Combinações campo+bacia sem match: 41
VOIP sem match: 34,938,929,059 bbl (3.3% do total)

```

- **Fact_economico:** para a tabela de fatos econômicos, verifiquei apenas uma lacuna decorrente da diferença entre início dos dados de Brent e Câmbio, mas o resultado apontou que isso não implicaria na análise proposta.

```

21

cambio_mensal = (df_cambio_tipado
    .withColumn("ano", F.year("data"))
    .withColumn("mes", F.month("data"))
    .groupBy("ano", "mes")
    .agg(F.avg("cambio_usd_brl").alias("cambio_usd_brl_medio")))

brent_mensal = (df_brent_tipado
    .withColumn("ano", F.year("data"))
    .withColumn("mes", F.month("data"))
    .groupBy("ano", "mes")
    .agg(F.avg("preco_brent_usd").alias("preco_brent_usd_medio")))

fact_economico = (cambio_mensal
    .join(brent_mensal, on=["ano", "mes"], how="outer")
    .withColumn("tempo_id", (F.col("ano") * 100 + F.col("mes")).cast("int"))
    .select("tempo_id", "cambio_usd_brl_medio", "preco_brent_usd_medio")
    .orderBy("tempo_id"))

# 6c. Validação: quantifica a lacuna esperada (meses só com uma das duas séries)
print(f"fact_economico: {fact_economico.count()} linhas")
qtd_lacuna = fact_economico.filter(F.col("cambio_usd_brl_medio").isNull() | F.col("preco_brent_usd_medio").isNull()).count()

```

Com as tabelas de fato e dimensão organizadas, salvei os dados.

```

23

tabelas_gold = ["dim_campo", "dim_poco", "dim_tempo", "fact_producao_mensal", "fact_reservas_anual", "fact_economico"]

for tabela in tabelas_gold:
    spark.sql(f"DROP TABLE IF EXISTS {CATALOGO}.{SCHEMA}.{tabela}")

dim_campo.write.mode("overwrite").saveAsTable(f"{CATALOGO}.{SCHEMA}.dim_campo")
dim_poco.write.mode("overwrite").saveAsTable(f"{CATALOGO}.{SCHEMA}.dim_poco")
dim_tempo.write.mode("overwrite").saveAsTable(f"{CATALOGO}.{SCHEMA}.dim_tempo")
fact_producao_mensal.write.mode("overwrite").saveAsTable(f"{CATALOGO}.{SCHEMA}.fact_producao_mensal")
fact_reservas_anual.write.mode("overwrite").saveAsTable(f"{CATALOGO}.{SCHEMA}.fact_reservas_anual")
fact_economico.write.mode("overwrite").saveAsTable(f"{CATALOGO}.{SCHEMA}.fact_economico")

for tabela in tabelas_gold:
    n = spark.table(f"{CATALOGO}.{SCHEMA}.{tabela}").count()
    print(f"{tabela}: {n} linhas")

```

> [See performance \(18\)](#) ✓ Checked for improvements

dim_campo: 590 linhas
dim_poco: 31304 linhas
dim_tempo: 1016 linhas
fact_producao_mensal: 5113367 linhas
fact_reservas_anual: 2130 linhas
fact_economico: 473 linhas

6) Análise

Escolhi fazer a análise das perguntas em um notebook próprio, lendo as delta table já criadas na Gold. O resultado está apresentado na seção seguinte.

```

2

from pyspark.sql import functions as F

CATALOGO = "PUC_Sprint_2"
SCHEMA = "anp"

fact_producao = spark.table(f"{CATALOGO}.{SCHEMA}.fact_producao_mensal")
fact_reservas = spark.table(f"{CATALOGO}.{SCHEMA}.fact_reservas_anual")
fact_economico = spark.table(f"{CATALOGO}.{SCHEMA}.fact_economico")
dim_campo = spark.table(f"{CATALOGO}.{SCHEMA}.dim_campo")
dim_poco = spark.table(f"{CATALOGO}.{SCHEMA}.dim_poco")
dim_tempo = spark.table(f"{CATALOGO}.{SCHEMA}.dim_tempo")

```

> [See performance \(1\)](#) ✓ Checked for improvements

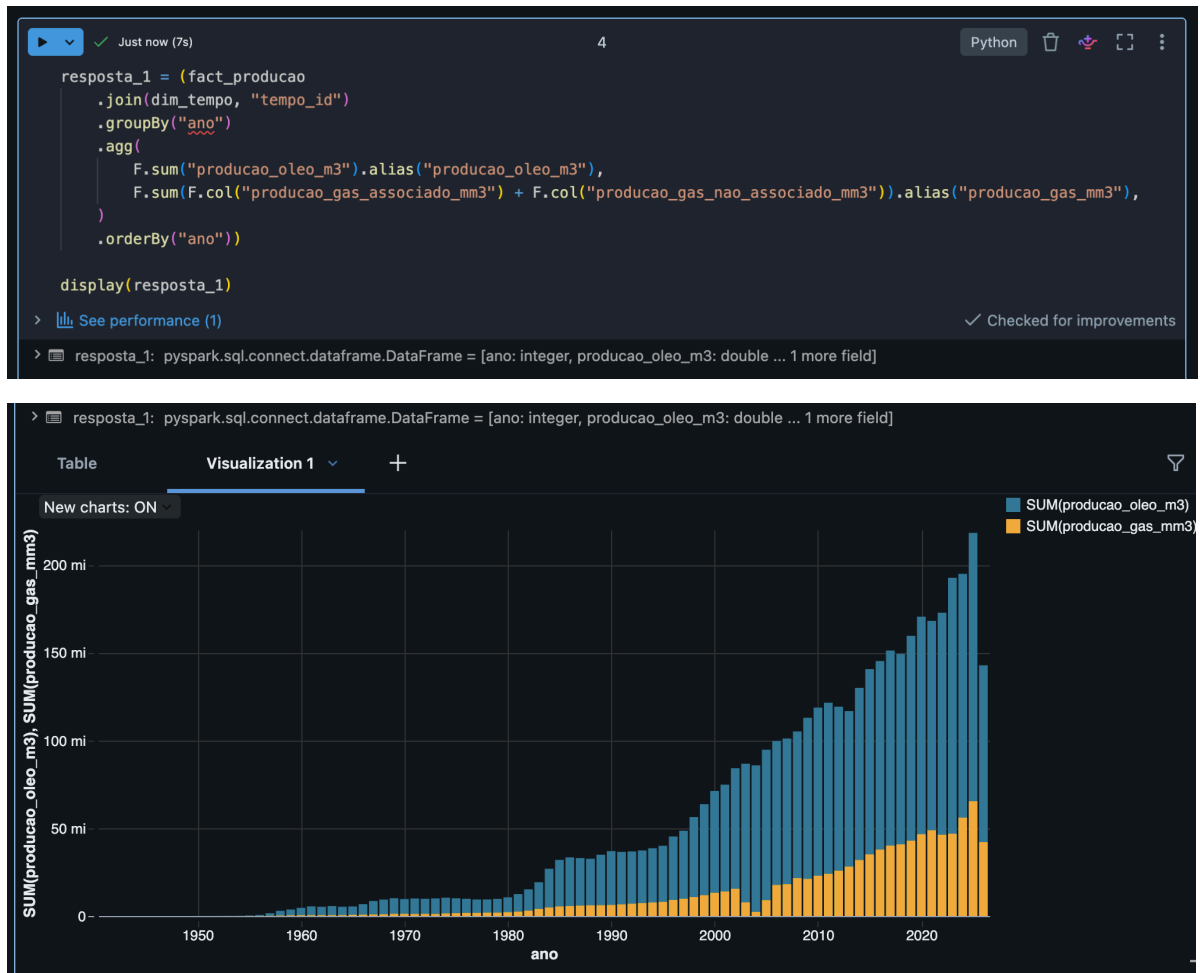
> 6 dataframes: fact_producao, fact_reservas, fact_economico, dim_campo, di...

5. Respostas e comentários

A) Qual foi a produção total de óleo e gás natural por ano?

Somei as colunas de produção da tabela `fact_producao_mensal` agrupando pelo ano, vindo de `dim_tempo`. Gás precisou juntar duas colunas, associado e não associado, já que o BMP não tem uma coluna única para isso, e mantive óleo e gás em eixos separados no gráfico por causa da diferença de unidade, m^3 contra milhões de m^3 . Usei "display" para melhor visualização.

Resultado ilustra o crescimento constante da produção no país. Os dados de 2026 ainda são parciais, o que explica a diminuição.



B) Quais os 10 campos com maior produção acumulada?

Join entre a `fact_produção` com `dim_campo`, agrupei por campo e bacia, já que nome de campo sozinho não é único, e somei a produção de óleo de cada um, ordenando do maior para o menor.

Resultado indica alta produção nos campos de Marlim, Tupi e Roncador, localizados no Pré-Sal.

```
resposta_2 = (fact_producao
    .join(dim_campo, "campo_id")
    .groupBy("campo", "bacia")
    .agg(
        F.sum("producao_oleo_m3").alias("producao_oleo_m3_acumulada"),
        F.sum(F.col("producao_gas_associado_mm3") + F.col("producao_gas_nao_associado_mm3")).alias(
            "producao_gas_mm3_acumulada"),
    )
    .orderBy(F.col("producao_oleo_m3_acumulada").desc())
display(resposta_2)
```

> [See performance \(1\)](#) ✓ Checked for improvements

	A _C campo	A _C bacia	1.2 producao_oleo_m3_acumulada	1.2 producao_gas_mm3_acumulada
1	MARLIM	CAMPOS	427814963.4924991	32571280.95479991
2	TUPI	SANTOS	323128393.9315004	96967093.60615012
3	RONCADOR	CAMPOS	259368282.66262034	34935143.92163997
4	MARLIM SUL	CAMPOS	239241995.73674127	26796579.567220066
5	LULA	SANTOS	222032371.71580023	59987907.146479994
6	JUBARTE	CAMPOS	158338354.85293013	21646822.87412001
7	ALBACORA	CAMPOS	148843959.60548058	15732926.703000024
8	BUZIOS_ECO	SANTOS	141613861.28841007	37642133.79757998
9	SAPINHOA	SANTOS	134978950.44369	35951979.70437004
10	BUZIOS	SANTOS	118781532.30104986	30036954.464489974
11	BARRACUDA	CAMPOS	107886450.8187098	9122453.637580018
12	MERO	SANTOS	94595278.84892003	39209897.56273997
13	MARLIM LESTE	CAMPOS	84439965.5184301	9107333.593619965
14	ALBACORA LESTE	CAMPOS	70115640.70909005	7447896.631239994

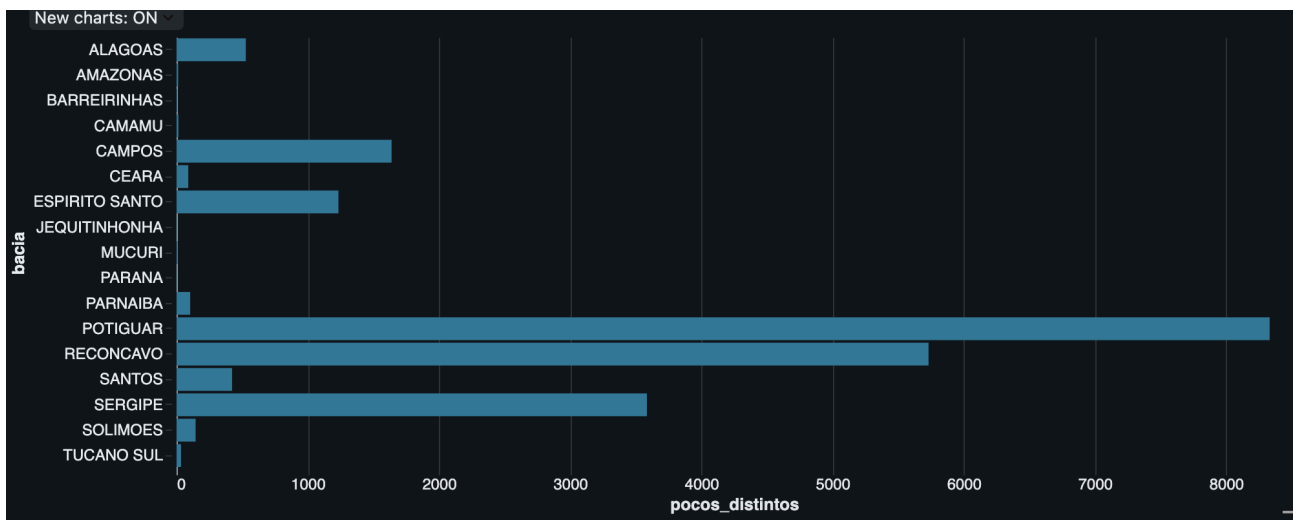
C) Qual a distribuição de poços por bacia?

A bacia de cada poço não está diretamente na tabela do BDEP de um jeito confiável, segui o caminho poço até campo até bacia, usando o join já validado entre a fato e dim_campo, e contei poços distintos por bacia.

Comparando com a resposta B, percebe-se que os poços de mar (Marlim, Tupi) tem produção por poço maior que os campos de terra.

```
resposta_3 = (fact_producao
    .join(dim_campo, "campo_id")
    .groupBy("bacia")
    .agg(F.countDistinct("poco_id").alias("pocos_distintos"))
    .orderBy(F.col("pocos_distintos").desc())
display(resposta_3)
```

> [See performance \(1\)](#) ✓ Checked for improvements



- D) Existe alguma correlação entre aumento do Brent e subida da produção? É possível avaliar o impacto 3 meses depois?

Agreguei a produção de óleo por mês e cruzei com o Brent médio mensal. Para simular a defasagem, usei uma função de janela que traz o valor do Brent de 3 meses atrás, e comparei a correlação com e sem essa defasagem.

A resposta indica que há correlação entre subida do Brent e subida da produção, mas não houve variação entre meses, o que pode ser explicado pela inércia da indústria do petróleo, já que não é possível abrir novos poços ou investir rapidamente.

```

from pyspark.sql import Window

producao_mensal_total = (fact_producao
    .groupBy("tempo_id")
    .agg(F.sum("producao_oleo_m3").alias("producao_oleo_total")))

janela_tempo = Window.orderBy("tempo_id")
LAG_MESES = 3

base_correlacao = (producao_mensal_total
    .join(fact_economico.select("tempo_id", "preco_brent_usd_medio"), "tempo_id")
    .orderBy("tempo_id")
    .withColumn("brent_defasado_3m", F.lag("preco_brent_usd_medio", LAG_MESES).over(janela_tempo)))

resposta_6 = base_correlacao.select(
    F.corr("producao_oleo_total", "preco_brent_usd_medio").alias("correlacao_sem_defasagem"),
    F.corr("producao_oleo_total", "brent_defasado_3m").alias("correlacao_defasagem_3_meses"),
)

```

correlacao_sem_defasagem	correlacao_defasagem_3_meses
0.6910936995912157	0.6862759878785218

- E) Qual seria a receita bruta estimada (produção × Brent × câmbio) por campo e por bacia?

Converti a produção de óleo de m³ para barris usando o fator padrão da indústria e multipliquei pelo Brent médio mensal e pelo câmbio USD/BRL, chegando a uma receita estimada em reais.

O resultado complementa a resposta B, com a receita concentrada nos poços do pré-sal.

```
Interrupt 00:05 15 Python
BARRIS_POR_M3 = 6.2898 # fator de conversão padrão da indústria: 1 m³ de óleo = 6,2898 barris

producao_mensal_valorada = (fact_producao
    .join(dim_tempo, "tempo_id")
    .join(dim_campo, "campo_id")
    .join(fact_economico.select("tempo_id", "preco_brent_usd_medio", "cambio_usd_brl_medio"), "tempo_id")
    .withColumn("producao_oleo_bbl", F.col("producao_oleo_m3") * F.lit(BARRIS_POR_M3))
    .withColumn("receita_estimada_brl", F.col("producao_oleo_bbl") * F.col("preco_brent_usd_medio") * F.col(
        "cambio_usd_brl_medio"))

resposta_8_por_campo_ano = (producao_mensal_valorada
    .groupBy("campo", "bacia", "ano")
    .agg(F.sum("receita_estimada_brl").alias("receita_estimada_brl"))
    .orderBy(F.col("receita_estimada_brl").desc()))

resposta_8_por_bacia_ano = (producao_mensal_valorada
    .groupBy("bacia", "ano")
    .agg(F.sum("receita_estimada_brl").alias("receita_estimada_brl"))
    .orderBy("ano", F.col("receita_estimada_brl").desc()))

resposta_8_por_campo_ano.show(20, truncate=False)
resposta_8_por_bacia_ano.show(20, truncate=False)
```

campo	bacia	ano	receita_estimada_brl
TUPI	SANTOS	2022	1.585245530773312E11
TUPI	SANTOS	2021	1.2533949009145982E11
TUPI	SANTOS	2023	1.2516992306031831E11
TUPI	SANTOS	2024	1.2343523413875978E11
TUPI	SANTOS	2025	1.1094538875005351E11
LULA	SANTOS	2019	8.773602461843658E10
LULA	SANTOS	2018	8.206144134770781E10
BUZIOS_ECO	SANTOS	2022	7.920080136922148E10
BUZIOS_ECO	SANTOS	2025	7.808430836407976E10
BUZIOS_ECO	SANTOS	2024	7.470335269747852E10
MERO	SANTOS	2025	7.410025755063567E10
TUPI	SANTOS	2020	7.40601718406348E10
BUZIOS_ECO	SANTOS	2023	6.917623611127663E10
BUZIOS	SANTOS	2021	5.528567435519908E10
LULA	SANTOS	2017	4.7075722909603424E10
MERO	SANTOS	2024	4.698919774687269E10

6. Autoavaliação

O projeto do MVP permitiu aprender como fazer data scraping de dados públicos, familiarizar-se com o ambiente do Databricks e melhorar a utilização do Python e do SQL. Creio que os resultados foram satisfatórios e que o modelo apresentado permite análises mais complexas, como avaliação temporal de poços individuais e a análise de volume e rendimento de poços maduros. Tais possibilidades podem ser exploradas em MVPs futuros.